

⚡ Essential Graph Search & Pathfinding Algorithms

🏠 Architectural Comparison & Walkthrough for UCS, GBFS, A*, and Bellman-Ford

👤 Author: Laurie MAVOUNGOU (CEO JK AI) | ✉️ lmavoungou@outlook.be

🟢 1. Uniform Cost Search (UCS)

UNINFORMED / COST-OPTIMAL Guarantees shortest path for non-negative edge costs using actual path cost $g(n)$.

🔥 **Core Mechanism:** Expands nodes using a Min-Heap Priority Queue sorted strictly by cumulative path cost $g(n)$.

🔧 Step-by-Step Execution:

- Initialization:**
 - Instantiate a tie-breaker counter using `itertools.count()` to handle priority queue comparisons cleanly.
 - Push start tuple to Min-Heap: `(0, next(counter), start, [start])`.
- Priority Queue Loop:**
 - Pop minimal element: `(cost, counter, current, path)`.
 - Visited Check:** Skip if `current` in `visited`; else add to `visited`.
 - Goal Check:** If `current == goal`, return optimal `path`.
- Neighbor Expansion:**
 - For each neighbor `n`, retrieve weight `w = weight(current, n)`.
 - Compute total path cost: $g(n) = g(current) + w$.
 - Push `(g(n), next(counter), n, path + [n])` into Min-Heap.

💡 **Pro-Tip:** Pass `(current, n)` to edge weight functions—not `(start, n)`!

🟣 2. Greedy Best-First Search (GBFS)

HEURISTIC / FAST SEARCH Focuses purely on estimated distance to goal $h(n)$ to speed up search speed.

Evaluation Function: $f(n) = h(n)$

🔧 Execution & Heuristic Logic:

- Coordinate Setup:** Maintain spatial coordinates for nodes (e.g., `A = (0,0)`) to evaluate straight-line heuristic distance.
- Heuristic Evaluation:** For candidate neighbor `n`, compute Euclidean distance:
$$h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$
- Greedy Selection:** Choose the frontier node with the lowest $h(n)$ to expand next.
- Path Tracking & Re-visitation:** Store predecessor pointers. If revisiting a node with a lower $h(n)$, update its cost and predecessor pointer.

⚠️ **Trade-off:** Highly efficient in search speed, but *not guaranteed* to yield the shortest overall path!

★ 3. A* Search Algorithm

INFORMED / COMPLETE & OPTIMAL Combines past cost $g(n)$ and heuristic estimate $h(n)$ for optimal pathfinding.

$f(n) = g(n) + h(n)$

🔧 Complete Walkthrough & Mechanics:

- Cost Accounting Variables:**
 - $g(n)$: Exact cost traveled from start node to `n`.
 - $h(n)$: Estimated Euclidean distance from `n` to goal.
 - $f(n)$: Total estimated cost of path through `n`.
- Iterative Expansion Loop:**
 - Initialize $g(start) = 0$ and compute $f(start) = h(start)$.
 - Pop node with lowest $f(n)$ value from priority queue.
 - For each neighbor `n` of `current`, calculate candidate step:
$$g_{new}(n) = g(current) + cost(current, n)$$
- Path Optimization & Relaxation:**
 - If $g_{new}(n) < g(n)$ (found a cheaper path to `n`):
Update $g(n) = g_{new}(n)$, set predecessor, and recompute $f(n) = g(n) + h(n)$.

🎯 **Admissibility Rule:** As long as $h(n)$ never overestimates true cost, A* guarantees finding the absolute optimal path.

🌟 4. Bellman-Ford Algorithm

DYNAMIC PROGRAMMING Single-source shortest path supporting negative edge weights and cycle detection.

🔧 Core 2-Phase Algorithm Flow:

- Initialization:**
 - Set $dist[start] = 0$, and $dist[v] = \infty$ for all `v` $\neq$ `start`.
 - Retrieve vertex count `V` and full edge collection `E`.
- Phase 1 — Relaxation Sweep (`V - 1` passes):**
 - Repeat loop `V - 1` times to guarantee propagation across all simple paths:
 - For every edge `(u, v)` with weight `w`:
$$\text{if } dist[u] + w < dist[v]: dist[v] = dist[u] + w$$
 - Update predecessor tracking map: `predecessor[v] = u`.
- Phase 2 — Negative Cycle Verification:**
 - Execute one extra pass across all edges `(u, v, w)`.
 - If $dist[u] + w < dist[v]$ still holds $\rightarrow$ **Flag Negative Weight Cycle!**

🛡️ **Guard Condition:** Check `if dist[u] != inf` before adding weights to avoid infinity arithmetic overflows!